

Low-Cost IEEE 802.11 and Bluetooth Attack Surfaces on Commodity Microcontrollers: A Case Study of the ESP32 Platform

Sameer Al Sahab

Department of Computer Science and Engineering
East West University, Dhaka, Bangladesh
Email: 2026-2-60-016@std.ewubd.edu

Abstract—The ESP32 is a five-dollar system-on-chip with a Wi-Fi and Bluetooth radio, and it gives firmware developers direct access to raw 802.11 frame injection and BLE advertising – capabilities that used to require a laptop and a monitor-mode adapter. This has led to a small ecosystem of open-source firmware projects that turn the chip into a pocket-sized wireless attack tool. This paper looks at one of them, Hydra-ESP, as a way of walking through why these attacks work at all. We cover the 802.11 and Bluetooth/BLE background needed to understand deauthentication, rogue access points, handshake and PMKID capture, beacon/probe flooding, and BLE proximity and HID spoofing, then close with the countermeasures that actually address the root cause: 802.11w, WPA3-SAE, rogue-AP detection, and tighter Bluetooth pairing policy. We deliberately stop short of reproducing exploit code – the goal here is to understand the attack surface well enough to defend against it, not to hand out a run-book.

Index Terms—IEEE 802.11, Wi-Fi security, deauthentication attack, evil twin, PMKID, WPA2/WPA3, Bluetooth Low Energy, BLE spoofing, ESP32, wireless intrusion detection

I. INTRODUCTION

Wi-Fi and Bluetooth security research used to mean a laptop, a software-defined radio or a monitor-mode USB adapter, and a full OS to drive it. That’s changed. Cheap 32-bit microcontrollers with built-in 802.11 and Bluetooth radios – the ESP32 being the obvious example – can now run an entire attack-and-monitoring stack off a battery pack you could fit in a pocket. Open-source projects like risinek’s *esp32-wifi-penetration-tool* [1] and Spacehuhn’s ESP8266/ESP32 deauther line [2] started this trend, and a number of derivatives have since layered on Bluetooth Low Energy spam and HID-injection modules on top. The result is genuinely useful for penetration testers and hobbyists learning wireless security, but the same accessibility means it’s just as easy to misuse against a network you don’t have permission to touch.

This paper looks at **Hydra-ESP**, an ESP32 firmware built on the risinek codebase, extended with a redesigned web UI, multi-target deauthentication, an on-device deauth-flood detector, optional OLED status output, and a handful of additional Wi-Fi and BLE attack modules. Rather than treat the firmware as something to be operated, we treat it as a map: each module corresponds fairly directly to a specific weakness in legacy 802.11 management frames, the WPA2 handshake, or BLE’s unauthenticated advertising channel. Walking through

what each module actually exploits, and why, sets up a defense discussion that isn’t specific to this one tool.

The rest of the paper goes: Section II covers the 802.11 and Bluetooth/BLE background needed to follow the rest of the paper. Section III describes the firmware’s architecture at a high level. Section IV goes through each attack category. Section V discusses a limitation that matters a lot in practice – the ESP32 only has one radio, and that constrains what the firmware can do at any given moment. Section VI covers detection and mitigation, and Section VII wraps up.

A note on scope. Consistent with normal responsible-disclosure practice for offensive security writing, this paper describes attack mechanisms at the protocol level – which frames are involved, why the protocol allows the behavior, what the attacker actually gains – without reproducing source code or step-by-step instructions. If you’re using firmware like this to test something, do it only on networks and devices you own or are explicitly authorized to test; unauthorized use runs into the Computer Fraud and Abuse Act (US), the Computer Misuse Act (UK), the ICT Act (Bangladesh), and equivalent laws almost everywhere else.

II. BACKGROUND

A. 802.11 Management Frames

802.11 splits traffic into data, control, and management frames. Management frames – beacons, probe requests and responses, authentication/association requests, and deauthentication/disassociation notices – handle network discovery and the process of joining, keeping, and leaving an association with an access point [3].

Two things about legacy management frames matter for everything that follows. First, they carry no confidentiality or integrity protection by default. Unlike data frames once WPA2/WPA3 encryption kicks in, management frames go out in the clear and aren’t authenticated, so any radio within range can build a syntactically valid frame claiming to come from any MAC address it likes. Second, a deauthentication or disassociation frame isn’t a request the receiver gets to negotiate – it’s a one-way notice, and the standard says a station is supposed to act on it immediately and drop the association, no matter who actually sent it.

The 802.11w-2009 amendment (Management Frame Protection, or MFP) later added cryptographic protection to a subset of these frames, including deauth and disassoc, once a station has completed a handshake with an AP that supports it [4]. MFP is mandatory under WPA3-Personal and optional in transitional WPA2/WPA3 setups [5].

B. The WPA2 Handshake and PMKID

WPA2-Personal derives a fresh session key (the Pairwise Transient Key) through a 4-way EAPOL handshake between the station and the AP, both of which already hold a shared Pairwise Master Key derived from the passphrase and SSID via PBKDF2 [6]. That handshake travels over the air. While the resulting key stays secret, the handshake itself carries enough information (ANonce, SNonce, a MIC) that someone who captures all four frames can later try an offline dictionary or brute-force attack against the passphrase, checking each guess against the captured material without needing to talk to the AP again.

A faster route targets the PMKID instead – an optional field some APs stick into the very first EAPOL frame to support fast roaming:

$$\text{PMKID} = \text{HMAC-SHA1-128}(\text{PMK}, \text{"PMK Name"} \parallel \text{AP_MAC} \parallel \text{STA_MAC}) \quad (1)$$

Because this can be pulled just by associating with the AP – no connected client required, and sometimes not even a full handshake – its disclosure enables the same offline dictionary attack without ever having to force a real client to reconnect [7].

Both the captured handshake and the PMKID typically get saved as .pcap and converted into a hash format (.hccapx, or Hashcat mode 22000) that offline cracking tools like Hashcat and Aircrack-ng understand [8].

C. Rogue Access Points

An *evil twin* is an attacker-run AP broadcasting the same SSID as a real network, usually with no password, so that nearby clients – which mostly rank networks by signal strength and SSID rather than any cryptographic check of who’s actually running them – end up connecting to it instead of (or after being kicked off) the legitimate AP [9]. 802.11 has no built-in way to tie an SSID to a specific operator, so this kind of impersonation is trivial at the protocol level. A captive portal on the clone then asks for the network password under some believable pretext, like a fake firmware update screen.

A related trick, sometimes called BSSID cloning or twin-deauth, goes a step further and also spoofs the real AP’s BSSID and channel, so the two APs look identical to clients. The resulting address conflict makes most client drivers drop the connection on their own. Because this works through address collision rather than an unprotected deauth frame, it can still work against clients running 802.11w/MFP, which is exactly the scenario where plain forged-frame deauth fails.

D. Beacons and Probe Requests

Beacons are the periodic broadcasts an AP uses to announce itself; probe requests are what a station sends to look for nearby APs, historically including the SSIDs of networks it’s joined before and is willing to reconnect to automatically. Neither is authenticated. That opens up two different tricks: flooding the area with beacons carrying made-up SSIDs to clutter every nearby device’s network list (beacon spam), and passively listening for the SSIDs a device asks about in its probe requests, then impersonating one of its already-trusted networks – an approach that goes back to the original KARMA attack [10]. Most phones now randomize their probe-request MAC address and probe far less actively than they used to, which cuts down this exposure but doesn’t remove it entirely.

E. BLE Advertising and HID Pairing

BLE peripherals advertise themselves on three fixed channels using unauthenticated packets that can carry manufacturer-specific data. Apple, Google, and Samsung all use that manufacturer field to drive their proximity UX – the AirPods lid-open animation, a Galaxy Buds pairing sheet, an Apple TV setup prompt, and so on. Because that advertisement format is just sitting there in the open air, and the receiving device’s UI reacts purely to what’s in the payload with no real check that the sender is an actual owned accessory, anything nearby that broadcasts the same manufacturer-data pattern can trigger the same prompts on devices it has nothing to do with. We’ll call this BLE proximity spam [11].

Separately, Bluetooth’s HID profile lets a peripheral register itself as a keyboard or mouse. Once a host pairs with something advertising the HID service, it accepts and processes whatever keystrokes that device sends. This is the same trust gap that USB BadUSB devices and other keystroke-injection attacks exploit: the host decides whether to trust the peripheral at pairing time, and once pairing succeeds it generally has no way to tell a real keyboard from anything else claiming to be one.

III. HYDRA-ESP: ARCHITECTURE OVERVIEW

Hydra-ESP is built on Espressif’s ESP-IDF for the ESP32 (Xtensa LX6, 802.11 b/g/n plus Bluetooth 4.2 BR/EDR and BLE). On top of the risinek foundation [1] it adds:

- a management access point and SPIFFS-hosted web UI for scanning, picking targets, configuring attacks, and watching live status;
- a modular Wi-Fi attack subsystem covering deauth, handshake capture, PMKID capture, beacon spam, probe/“Ghost Mode” spam, evil twin, BSSID clone, and SSID cloning;
- a BLE/BT subsystem for proximity-spam advertising and a Bluetooth HID keyboard payload;
- a promiscuous-mode deauth detector that watches for attack signatures in management traffic;
- optional SSD1306 OLED status output.

Each Wi-Fi module runs the radio in one of two modes. Injection mode constructs raw 802.11 management frames

directly through ESP-IDF's `esp_wifi_80211_tx` call, sidestepping the normal association state machine entirely – that's how deauth and handshake capture work. Soft-AP mode has the firmware stand up its own access point, which is how evil twin, BSSID clone, and beacon spam operate. The BLE modules mirror this split using NimBLE, either in raw GAP-advertising mode for proximity spam or GATT/HID-server mode for the keyboard payload. This two-mode split tracks the two attacker capabilities from Section II pretty closely: forging unauthenticated management signaling, and standing up an unauthenticated endpoint of your own.

A. Bypassing the Vendor Wi-Fi Stack's Own Safety Check

Injection mode is worth a closer look, because it doesn't work purely through public API calls. Espressif ships the ESP32's low-level Wi-Fi stack (the "WSL", short for Wi-Fi Stack Libraries) as a closed-source binary blob, and that blob includes an internal sanity-check routine that inspects outgoing raw frames and refuses to transmit certain management-frame types – deauthentication among them – as a vendor-level guard rail against exactly this kind of misuse. Hydra-ESP inherits a bypass for this from the risinek codebase: at link time, it deliberately defines a function with the same symbol name as the blob's internal check and links with a flag that permits duplicate symbol definitions, so the linker resolves calls to that check against the firmware's own replacement, which unconditionally reports every frame as valid. The result is that raw frame injection, once blocked by the vendor SDK, goes through unfiltered. This detail matters for the defense discussion in Section VI: it shows that the manufacturer already anticipated and tried to guard against this misuse at the SDK level, and the attack firmware works specifically by defeating that guard, not by exploiting the underlying radio hardware itself.

IV. ATTACK ANALYSIS

A. Deauthentication and Disassociation Flooding

By sending forged deauth or disassociation frames with the target AP's BSSID as the claimed source, the firmware makes targeted clients (or, with a broadcast address, everyone) drop their connection – straight exploitation of the unauthenticated-control-frame problem from Section II-A. Hydra-ESP can target up to sixteen clients at once and offers deauth-only or deauth-plus-disassoc modes. As you'd expect, clients on an 802.11w/MFP-enabled WPA3-SAE network are largely unaffected, since the forged frame can't produce the per-session MIC a protected frame needs.

B. WPA2 Handshake and PMKID Capture

The handshake module can optionally force a reconnection with a short deauth burst against an already-connected client, then quietly records the resulting 4-way EAPOL exchange to `.pcap/.hccapx` for later offline work, following the mechanism in Section II-B. The PMKID module skips the client entirely – it just associates with the target AP and checks whether the first EAPOL message includes the optional

PMKID field. Either way, the firmware itself doesn't crack anything; it only grabs the material a separate offline step would need, run on different hardware.

C. Evil Twin

This one is more deliberate than a blind capture-and-hope loop. The module stands up an open clone AP using the target SSID while running a deauth cycle against the real AP – but as soon as a client actually joins the clone, it pauses the deauth (there's no reason to keep kicking a client that's already on your fake network). The captive portal then collects whatever password the user submits. Here's the interesting part: it doesn't just log that password and move on. It tears down the portal, briefly connects to the *real* target AP as a client using the submitted password, and checks whether that connection succeeds. If the password is wrong, it's logged as a failed attempt, the portal comes back up, and the deauth cycle resumes. If it's correct, the attack stops and reports the verified password. That active verification step is what separates this from a captive portal that just harvests whatever gets typed in – it only reports a result once it's actually confirmed against the real network.

D. BSSID Clone (Twin-Deauth)

This variant additionally spoofs the legitimate AP's BSSID and channel, so the real AP and the clone look identical to clients. As covered in Section II-C, this displaces clients through address-conflict behavior baked into their own connection logic, not through a forged control frame – which is exactly why it still works against MFP/WPA3-hardened clients where plain deauth doesn't. A simpler sibling module, SSID Cloning, skips the BSSID spoofing and just stands up several APs sharing the target's SSID (distinguished only by trailing whitespace so they still display as the same name); it doesn't reliably disconnect anyone by itself but adds to the same confusion that beacon spam relies on.

E. Beacon Spam and Probe/"Ghost Mode" Harvesting

The beacon-spam module pushes out 1 to 100 beacon frames per configured interval, each with a randomized SSID, taking advantage of the lack of beacon authentication (Section II-D) to clutter nearby scan lists. Ghost Mode instead listens in promiscuous mode for SSIDs in nearby probe requests and re-broadcasts them as its own beacons – the probe-harvesting/KARMA approach from the same subsection. It's noticeably less effective against phones that no longer send directed probes for networks they've already joined.

F. BLE Proximity Spam

The BLE spam module advertises manufacturer-specific GAP payloads matching known Apple, Samsung, and Google proximity-pairing formats – AirPods, Galaxy Buds, Apple TV setup, and similar – which triggers the matching OS-level pairing prompt on any compatible device nearby, per Section II-E. No actual pairing or data exchange happens; the impact is limited to unwanted popups and, at high broadcast rates, some battery and CPU overhead on the receiving devices.

G. Bluetooth HID Payload

This module advertises the ESP32 as a Bluetooth HID keyboard under a randomized name. If a host is manually paired with it, the firmware sends a pre-programmed keystroke sequence – basically a Bluetooth version of a BadUSB device (Section II-E). Because Bluetooth HID pairing on most OSes still needs explicit user confirmation – a PIN, a passkey, or an “Allow” prompt – unless legacy just-works pairing is exposed, the real risk here comes down to social engineering someone through that pairing step, not a silent zero-click compromise. In this firmware the bundled payload options are illustrative rather than exhaustive: one is a harmless novelty (playing a sound), and the other opens a terminal and runs a short script that reads a Windows host’s saved Wi-Fi profiles (which Windows will reveal in cleartext through its own `netsh` utility for networks the machine has already joined) and posts the result back to the ESP32 over its own management-AP web server. It’s a useful illustration of how much damage keystroke injection alone can do without any separate malware payload – it just automates commands the operating system already exposes to any local user.

H. Deauth Attack Detector

The one defensive module in the firmware puts the radio into promiscuous monitor mode and counts deauth frames per source BSSID in a rolling one-second window, flagging bursts above a threshold (more than 10/second) and any deauth frame using the reserved broadcast source `00:00:00:00:00:00` – both recognized signatures of an active deauth attack in prior wireless IDS work [12].

V. PLATFORM LIMITATIONS: WHAT A SINGLE RADIO CAN’T DO

It’s worth being honest about what the ESP32 can’t do, because it shapes the firmware’s design more than the feature list suggests. The chip has exactly one 2.4 GHz radio shared between Wi-Fi station, Wi-Fi AP, and Bluetooth/BLE modes – there’s no second radio to fall back on, and no 5 GHz support at all, so the whole attack surface here is limited to 2.4 GHz networks and devices.

That single-radio constraint is the reason the management web UI drops out during several attacks. Deauth, evil twin, BSSID clone, and beacon spam all need exclusive use of the radio – injecting raw frames or running a soft-AP at the target’s channel and, for the rogue-AP attacks, its BSSID – and the ESP32 can’t simultaneously keep serving its own management AP on a different channel while doing that. So for the duration of those attacks, the operator loses the web interface entirely. Hydra-ESP works around this with a configurable timeout that brings the management AP back automatically; without one set, the only way to regain control is a power cycle. This is a real usability weakness, not just an inconvenience – it means the deauth detector module, which also needs exclusive promiscuous-mode access to the radio, can’t run at the same time as any of the attack modules either. The firmware can attack or it can monitor, but not both at once.

The chip’s modest compute and memory budget (a dual-core Xtensa LX6 with a few hundred KB of RAM) also caps how much it can do at high packet rates – handshake and PMKID capture buffers are bounded by available RAM and SPIFFS flash space, so long capture sessions or very busy networks can run into storage limits that a laptop-based capture setup wouldn’t. And because it’s a single onboard PCB antenna rather than a purpose-built high-gain adapter, effective range is short compared to the equipment this kind of attack is normally associated with. None of this changes whether the underlying protocol weaknesses are real, but it does mean the ESP32 is a proof-of-concept-grade platform, not a substitute for dedicated wireless testing hardware.

VI. DETECTION AND MITIGATION

A. Protocol-Level Mitigations

Turning on 802.11w (Management Frame Protection) is the single most direct fix for the deauth technique in Section IV-A – it cryptographically authenticates deauth/disassoc frames, which the raw-injection attack simply can’t forge. Set it to *required*, not *optional*, wherever every client on the network actually supports it [5]. Moving to WPA3-SAE addresses the handshake/PMKID problem from Section IV-B directly, since SAE’s Diffie-Hellman-based exchange isn’t vulnerable to offline dictionary attack the way a captured WPA2 handshake or PMKID is [5]. On APs where legacy fast-roaming PMKID advertisement isn’t actually needed, disabling it removes the clientless capture path from Section II-B entirely.

B. Network and Operational Mitigations

Enterprise APs and dedicated wireless IDS sensors can run the same deauth-flood and rogue-BSSID checks described in Section IV-G, but at network scale, correlating signals across multiple APs to tell an actual attack apart from one misbehaving device. Comparing observed BSSID/SSID/channel fingerprints against a known-good baseline and alerting on unexpected duplicates catches both the plain evil-twin and BSSID-clone attacks from Sections IV-C and IV-D. On the human side, training people to check for HTTPS and a valid certificate, and to never type a Wi-Fi password into a browser page claiming to be a router update, cuts down the success rate of the credential-harvesting step regardless of which radio trick got them there in the first place. Strong, unique passphrases combined with WPA3 where it’s available also limit the damage even if a handshake or PMKID does get captured.

C. Bluetooth/BLE Mitigations

Turning off Bluetooth discoverability when it’s not needed, and treating unexpected pairing prompts as suspicious by default, covers most of the proximity-spam nuisance from Section IV-E. Requiring explicit confirmation for HID pairing – and where it’s organizationally feasible, restricting which device classes are allowed to pair with managed endpoints – addresses the BadUSB-style risk in Section IV-F. Keeping Bluetooth stacks updated matters too, since OS vendors have

periodically tightened how they validate proximity-pairing payloads in response to exactly this kind of spam.

VII. CONCLUSION

What makes Hydra-ESP a useful case study is how closely its module boundaries track the actual protocol weaknesses. Every attack it implements comes down to one of two things: the lack of authentication on 802.11 management frames and BLE advertisements, or a trust decision – associating with an SSID, accepting a PMKID field, pairing with a HID device – that client software still makes without any real cryptographic identity check. None of this is new or specific to this firmware; it’s the same set of gaps that 802.11w, WPA3, and modern OS-level pairing confirmation were built to close. That’s also why the defenses in Section VI work: they target the actual protocol gap rather than any specific tool, so they hold up against the entire category of cheap ESP32-class attack platforms, not just the one covered here.

REFERENCES

- [1] risinek, “esp32-wifi-penetration-tool,” GitHub repository, [Online]. Available: <https://github.com/risinek/esp32-wifi-penetration-tool>
- [2] Spacehuhn Technologies, “esp8266_deauther,” GitHub repository, [Online]. Available: https://github.com/SpacehuhnTech/esp8266_deauther
- [3] IEEE Standard for Information Technology—Telecommunications and Information Exchange between Systems, “IEEE Std 802.11-2020,” IEEE, 2021.
- [4] IEEE Std 802.11w-2009, “Amendment 4: Protected Management Frames,” IEEE, 2009.
- [5] Wi-Fi Alliance, “WPA3 Specification,” Version 3.x, Wi-Fi Alliance, 2020.
- [6] IEEE Std 802.11i-2004, “Amendment 6: Medium Access Control (MAC) Security Enhancements,” IEEE, 2004.
- [7] J. Steube, “New attack on WPA/WPA2 using PMKID,” Hashcat Forum, 2018.
- [8] Hashcat Project, “hashcat – advanced password recovery,” [Online]. Available: <https://hashcat.net/hashcat/>
- [9] K. Nakhila, A. Attiah, C. Jin, and C. Zou, “Parallel Active Dictionary Attack on WPA2-PSK Wi-Fi Networks,” in *Proc. IEEE MILCOM*, 2015.
- [10] D. Robinson, “KARMA attack,” presented at ShmooCon, 2005.
- [11] J. Wu, “BLE proximity-pairing spoofing analysis,” independent security research, 2022.
- [12] Y. Bulusu, R. Shetty, and S. Nadkarni, “Detecting 802.11 deauthentication attacks using traffic analysis,” in *Proc. IEEE Int. Conf. on Wireless Networks*, 2016.